Queueing Theory Fundamentals

why latency explodes near full utilization

Watch the 4-min video

In one line: Little's Law says items in a system = arrival rate × time in system ($L = \lambda W$); with random arrivals, time in system grows like service time ÷ (1 − utilization), so latency doubles at 50%, is 10× at 90% and 20× at 95%; run interactive services around 60–80%, alert on queues, and prefer shared queues over per-server queues.

1 From 70% to 90%, latency triples

Your servers run at 70% CPU and latency is fine. Traffic grows, CPU reaches 90%, and latency doesn't grow a little: it triples. Nothing is broken; this is simply how queues behave.

2 Little's Law

The average number of items in a system equals the arrival rate times the average time each spends inside: **$L = \lambda \times W$**. At 100 requests per second, each taking 200 ms, about **20 requests are in flight** on average. It holds for any stable system, whatever the distributions (Phase 1, lesson 9). Turned around, it lets you estimate latency from a queue length you can observe: $W = L / \lambda$.

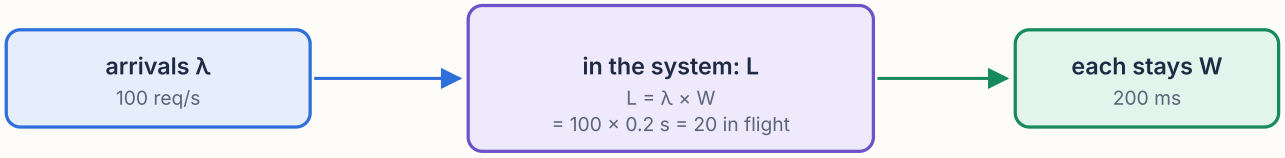


Figure 1. Little's Law with numbers: 100 requests per second × 0.2 s = 20 in flight.

3 Utilization and random arrivals

Utilization ρ is the arrival rate divided by the service rate: a server that can handle 100 requests per second and receives 80 is 80% utilised. But requests arrive **randomly**, not evenly spaced, so sometimes several arrive together and queue. The closer utilization gets to 100%, the less spare time the server has to clear those bursts.

4 The M/M/1 curve

The simplest model is **M/M/1**: random (Poisson) arrivals, random (exponential) service times, one server. Its key result:

```
time in system W = S / (1 − ρ)  (S = service time, ρ = utilization)
```

At 50%, time doubles. At 80%, it's 5×. At 90%, 10×. At 95%, 20×. The curve shoots up as utilization approaches 100%. Real systems with several servers or less variable service times do better, but the shape, a **knee** followed by a wall, is the same.

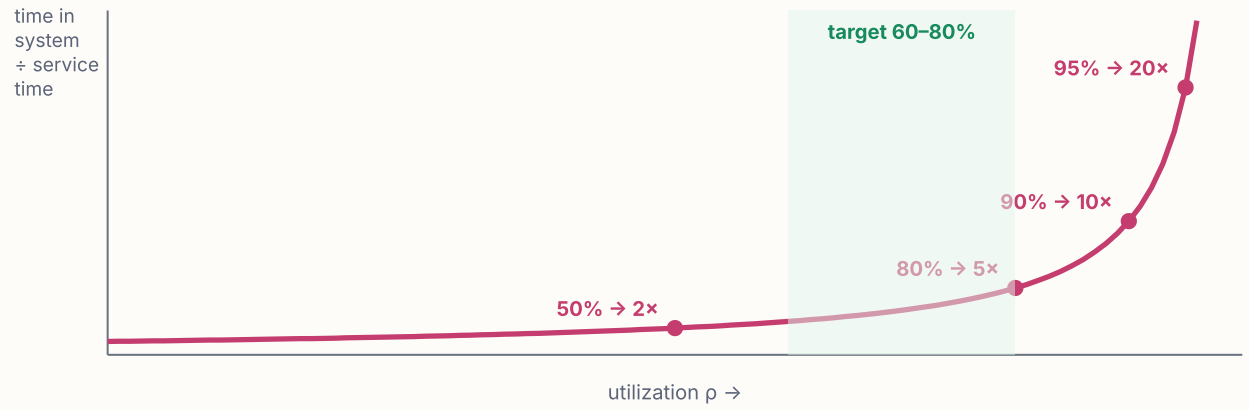


Figure 2. M/M/1: time in system as a multiple of service time. Below the knee latency barely moves; above it, it explodes.

5 Headroom isn't waste

That's why teams target around **60 to 80%** utilization for interactive services. Below the knee, small increases in traffic barely change latency. Above it, a small burst creates a long queue, latency explodes, and timeouts and retries add even more load (lesson 10). Batch jobs can run hotter, because nobody is waiting on them interactively.

WORKED EXAMPLE: WHY LATENCY TRIPLES

With a 10 ms service time: at 70% CPU, time in system ≈ 10 / 0.3 ≈ **33 ms**. At 90%, ≈ 10 / 0.1 = **100 ms**. Same code, same server, three times the latency, all from queueing.

6 Using it in practice

- **Size pools** with Little's Law: 500 requests per second × 40 ms of DB time each ≈ 20 connections busy on average; add headroom for bursts.
- **Alert on queue length and saturation**, not only CPU (lesson 42).
- **Autoscale before the knee** (lesson 14).
- **Share queues**: several servers pulling from one queue behave much better than separate queues per server, one reason load balancing (and least-connections) helps.

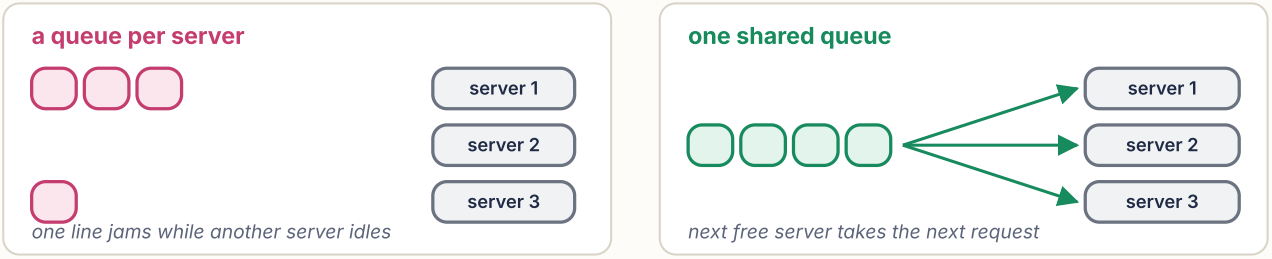


Figure 3. Separate queues let one line jam while a server idles; a shared queue keeps every server busy.

COMMON MISTAKES

- Planning capacity at 90%+ utilization for user-facing services.
- Assuming latency grows linearly with load.
- Ignoring variability: bursty arrivals or very uneven request costs make the knee come earlier.

7 Key terms

Little's Law: L = λ × W

Arrival rate λ: requests per second coming in

Service rate μ: requests per second a server can finish

Utilization ρ: λ / μ

M/M/1: random arrivals, random service, one server

Knee: where latency starts rising steeply

8 Interview questions

Q1. Why should you not run servers at 95% utilization? BASIC

Because waiting time grows like 1 / (1 − utilization), so near full load latency is about 20 times the service time, and any burst causes huge queues and timeouts.

Q2. What does Little's Law say? BASIC

Items in the system equal arrival rate times time in the system: L = λ × W.

Q3. A service gets 2,000 req/s and each request holds a DB connection for 15 ms. How big a pool? SCENARIO

L = 2,000 × 0.015 = 30 connections busy on average; size for bursts and the utilization target, e.g. 40 to 50, and check the database can sustain that concurrency.

Q4. Service time is 20 ms and you need p50 under 60 ms. Rough max utilization? SCENARIO

W = 20 / (1 − ρ) ≤ 60 → 1 − ρ ≥ 1/3 → ρ ≤ about 67%.

Q5. Why does one shared queue beat a queue per server? TRICKY

With separate queues, one server can be idle while another's queue is long; a shared queue lets any free server take the next item, cutting waiting time sharply at the same utilization.

Q6. Does Little's Law need Poisson arrivals? TRICKY

No. It holds for any stable system in the long run, regardless of arrival or service distributions; the 1 / (1 − ρ) formula is the part that assumes M/M/1.

Next: SD-J-013 · Tail Latency backendroadmap.com